

Solidity-to-Frontend Interaction & Browser Deployment: Open-Source Templates and a Reference Architecture for the PYTHAI/BANKON Ecosystem

TL;DR

- The "Etherscan login-to-contract" experience is **not a Hardhat feature** — it is the block explorer rendering a form from a contract's **verified ABI** plus an injected/WalletConnect provider; you can reproduce it open-source with **abi.ninja** (BuidlGuidl, MIT), **eth95** (adrianmcli, MIT), or **scaffold-eth-2's Debug Contracts** page, and you can fully match your Foundry-first, mainnet-only conventions without Hardhat.
- In-browser deployment of your **Foundry artifacts** (`out/Contract.sol/Contract.json`) → (`abi`) + (`bytecode.object`) is a solved pattern: ship the artifact to the page and call (`new ethers.ContractFactory(abi, bytecode, signer).deploy(...)`) or viem's (`walletClient.deployContract({abi, bytecode, args})`), then poll the receipt for the deployed address — no Hardhat, no proxies, no admin keys required.
- For AgenticPlace/BANKON, the recommended reference architecture is **vanilla JS + viem** (or **ethers v6**) + **EIP-6963 wallet discovery** + **a chain switcher driven by your CAIP-2** (`allchain.html`) **map** + **ContractFactory/deployContract** + **Blockscout verification API**, optionally token-gated by the x402-avm "Parsec" paywall on Algorand. CREATE2 via the Arachnid/Safe singleton factory gives you the same address across every EVM chain in your map.

Key Findings

1. The Etherscan pattern is an ABI-rendering trick, not a Hardhat trick

Your hunch deserves a direct answer: the "connect to Web3 and call Read/Write functions" UX has **nothing to do with Hardhat**. The mechanism is:

1. A contract's source is **verified** on the explorer (Etherscan/Blockscout) — the explorer recompiles the source with the declared compiler version/settings and matches the resulting bytecode to the on-chain bytecode.
2. Verification exposes the **ABI** on the explorer.
3. The explorer **renders an HTML form input per function** (one field per ABI input), grouped into Read (view/pure, free (`eth_call`)) and Write (state-changing, (`eth_sendTransaction`)).
4. "Connect to Web3" attaches an **injected EIP-1193 provider** (MetaMask et al.) or WalletConnect; Write calls are encoded from the form values against the ABI and submitted as a transaction.

Etherscan even lets you attach a **custom ABI** to any address (verified or not) for your own account, which proves the UI is purely ABI-driven. Blockscout implements the same thing — a React-based interface that auto-detects proxy patterns (EIP-1967/1822/Diamond) and renders Read/Write/Read Proxy/Write Proxy tabs once the contract is verified.

What Hardhat actually provides is a local node (`hardhat node`), a JS/TS console, a test runner, and deploy scripting (often via `hardhat-deploy`) — **no default UI** for contract interaction. The "auto-UI" you're thinking of comes from three distinct open-source sources, none of which are Hardhat itself:

- **Etherscan/Blockscout** (verified ABI → form, the canonical example).
- **scaffold-eth-2's "Debug Contracts" page** — auto-generates a UI from the ABI of contracts in your project (works with **either** Hardhat **or** Foundry as the Solidity framework).
- **abi.ninja** (BuidlGuidl) and **Remix's "Deployed Contracts" panel** — paste an address+ABI (or load a verified contract) and get a generated form.

So the pattern is reproducible regardless of your build tool. Since you use Foundry, scaffold-eth-2's Foundry flavor and the Wagmi CLI Foundry plugin give you the same outcome without touching Hardhat.

2. ABI-to-UI generators (reproduce the Read/Write experience as open source)

Repo	Stack	License	Notes / Activity
BuidlGuidl/abi.ninja	React, NextJS, RainbowKit, wagmi, viem (built on Scaffold-ETH 2)	MIT	The closest OSS clone of Etherscan's interaction UI for contract ABIs and source code directly using ethers.js. It decompiles unverified contracts via heimdall-rs; at localhost 31337. 204 stars / page; actively maintained. Interaction only — no code component.
adrianmclli/eth95	Vanilla front-end (Parcel) + Express, web3	MIT	"Instant retro UI for calling any contract function." artifacts or paste an ABI. Live at eth95.dev. Good for reader/writer.
hiddentao/ethereum-abi-ui	Framework-agnostic JS (jQuery example)	(npm <code>ethereum-abi-ui</code>)	Library that auto-generates form-field definitions from contract ABIs using <code>canRenderMethodParams</code> and <code>renderMethodParams</code> . Old (last publish ~8 yrs) but the algorithm is exact.

Repo	Stack	License	Notes / Activity
nikoumba/ethereum-smart-contract-interaction-tool (SpiralOutDotEu)	React + ethers + wagmi	open source	Upload ABI → call read/write → also generates a re ABI.
ArviinM/abi-ui	React, ethers, wagmi, react-hook-form, zod, tailwind/daisyui, vite	open source	Stores contracts in browser localStorage; clean sep ContractReader/ContractWrite components.
danielefavi/ethereum-interface-generator	CLI ((eth-ui-gen)) generating a GUI	open source	Generates a customizable interface from artifact J
cryptol0g1c/react-eth	React	MIT	<code><ReactEth abi={abi}></code> renders a form from a

3. Vanilla JS + ethers/viem wallet-connect contract interaction

Wallet discovery (EIP-6963, multi-wallet). The modern standard replaces the single `window.ethereum` slot with an event-based announce/request handshake, so users with several wallets get a proper picker instead of a "last-to-load wins" race. For vanilla JS, listen for `eip6963:announceProvider` and dispatch `eip6963:requestProvider`; the wevm **MIPD store** (`mipd`) wraps this for both vanilla JS and React. MetaMask's `vite-react-ts-eip-6963` is the canonical reference; web3.js ships `requestEIP6963Providers()`/`onNewProviderDiscovered()`. If you go React, wagmi v2's `injected()` connector does EIP-6963 discovery automatically.

Read/write. ethers v6 `BrowserProvider(window.ethereum) → getSigner() → new Contract(addr, abi, signer);` viem `createWalletClient({transport: custom(window.ethereum)}) + createPublicClient` for reads. viem's `readContract`/`writeContract`/`waitForTransactionReceipt` map cleanly onto the Read/Write tab split.

4. React references (if/when you want them)

- scaffold-eth-2** (`scaffold-eth/scaffold-eth-2`, MIT): NextJS + RainbowKit + wagmi + viem, **Foundry or Hardhat**. Contract Hot Reload publishes ABI+address to the frontend; custom hooks `useScaffoldReadContract`/`useScaffoldWriteContract`/`useScaffoldContract`; the

Debug Contracts page is the ABI-driven UI. Deployment is via `yarn deploy` (a backend script with a deployer key), **not** a browser click-to-deploy.

- **Wagmi CLI Foundry plugin** (`@wagmi/cli/plugins` → `foundry`): points at your Foundry `out/` dir, watches for changes, and (with the `react` plugin) generates **type-safe hooks** (`useReadX`, `useWriteX`, `usePrepareX`) directly from your Foundry artifacts. This is the cleanest Foundry→React bridge and supports `--watch`.
- Wallet kits: **RainbowKit**, **ConnectKit**, **Web3Modal**/**Web3Onboard** all support EIP-6963.

5. In-browser deployment of compiled artifacts (the "click to deploy from wallet")

The core call.

- ethers v6: `const factory = new ethers.ContractFactory(abi, bytecode, signer);`
`const c = await factory.deploy(...args); await c.waitForDeployment(); const`
`addr = await c.getAddress();` ethers also offers `ContractFactory.fromSolidity(compilerOutput, signer)` which consumes solc output directly.
- viem: `const hash = await walletClient.deployContract({abi, bytecode, args,`
`account}); const {contractAddress} = await`
`publicClient.waitForTransactionReceipt({hash});` (use `transport:`
`custom(window.ethereum)` for the browser wallet).

Feed it Foundry artifacts directly. A Foundry artifact at `out/Counter.sol/Counter.json` contains `abi` and `bytecode.object` (a `0x`-prefixed creation-bytecode hex string), plus `deployedBytecode`, `metadata`, and `linkReferences`. Ship that JSON to the page; `abi` → `ContractFactory/deployContract` ABI, `bytecode.object` → bytecode argument. (Note: a viem GitHub discussion documents that wagmi-generated bytecode sometimes needs to be replaced with the raw Foundry `bytecode.object` to deploy correctly — prefer the Foundry artifact's bytecode.)

Precompiled artifacts vs in-browser compilation. Two valid approaches:

- **Ship precompiled Foundry artifacts (recommended for you).** Deterministic, reproducible, matches your Foundry-first + mainnet-only conventions, and the bytecode you deploy is exactly what you tested. No 8 MB compiler in the browser.
- **Compile in-browser with solc-js** in a Web Worker (the only supported browser path — `importScripts('https://binaries.soliditylang.org/bin/soljson-...js')` + `solc/wrapper`), or the higher-level **gnidan/web-solc** (`fetchAndLoadSolc("^0.8.26")`), Worker-based, plus `@web-solc/react`, **zianksm/solc-browserify** (supports import callbacks), or `@agnostico/browser-solidity-compiler`. Use this only if users author/edit Solidity in-app (Remix-style). For shipping known contracts, precompiled artifacts win.

Constructor-argument form generation. Read the `constructor` entry in the ABI (`(type: "constructor"), (inputs[])`) and render one input per arg using the same type→field mapping as the Write tab (address/uint/bool/bytes/tuple). Pass collected values as `(...args)`.

Open-source deployer dApps and patterns.

- **CryptoFusion** (Azim Memon; live at cryptofusion.vercel.app, full source via the author's build guide at azimmemon2002.github.io) — the **closest exact match** to your target pattern: React + **wagmi v2** + **viem** + `useDeployContract` + `useWaitForTransactionReceipt`, paste ABI + bytecode (from `artifacts/.../CustomToken.json`) + token params, "Click Deploy and confirm in your wallet," chains from `wagmi/chains`. Verbatim: *"we compile the bytecode locally and deploy it directly from the frontend... No factory contract needed."* Caveat: it's a tutorial/portfolio project; I could not confirm a canonical standalone GitHub repo URL or an explicit license, so treat it as a reference implementation rather than a dependency.
- **thirdweb** (`thirdweb-dev/contracts`, and `thirdweb-example/custom-dashboard`): `sdk.deployer.deployBuiltInContract(...)` / `ContractDeployer` deploy prebuilt or custom contracts **from the connected wallet, no private keys**, with a generated contract dashboard (read/write/events). Open source; good for "deploy + auto-dashboard." Note thirdweb leans on its own infra and some upgradeable/proxy patterns — screen against your no-proxy/no-admin-key rule. [GitHub](#)
- **OpenZeppelin Contracts Wizard** (`OpenZeppelin/contracts-wizard`): interactive generator; embeddable via `<oz-wizard>`; "open in Remix" → deploy with injected wallet. OZ also now advertises a feature to "spin up user interfaces for any deployed contract... auto-generate a React UI with wallet-connect and multi-network support, and export a complete app." MIT.
- **eth95** doubles as a quick deployer when pointed at artifacts.
- **abi.ninja** and **scaffold-eth-2** do NOT ship a browser deploy-from-bytecode component — abi.ninja is interaction-only; scaffold-eth-2 deploys via a backend script. Don't expect a turnkey deploy button there.

6. Multi-chain deploy + chain switching against your CAIP-2 map

Switch/add chains. Use EIP-3326 `wallet_switchEthereumChain` (`(params: [{chainId: '0x...'}])`); on error code **4902** (chain unknown to the wallet), call EIP-3085 `wallet_addEthereumChain` with `{chainId, chainName, rpcUrls, nativeCurrency{name, symbol, decimals}, blockExplorerUrls}`, then retry the switch. This is a few lines of vanilla JS and is the exact primitive your `allchain.html` CAIP-2 registry should feed: map each `eip155:<id>` entry to the hex chainId + RPC + explorer + native currency, and generate the add/switch params from it. (Note: MetaMask historically required native currencies to use 18 decimals — relevant if any chain in your map differs.)

Deterministic "same address on every chain." For deploying the **same artifact across Base, Moonbeam, and your other EVM chains** at an identical address, use **CREATE2** via a singleton factory:

- **Arachnid/deterministic-deployment-proxy** — the classic proxy at `(0x4e59b44847b379578588920cA78FbF26c0B4956C)`. Per its README, it is deployed by a one-time-use account (signer `(0x3fab184622dc19b6109349b94811493bf2a45362)`) so "no matter what chain the deployer is on, its address will always be the same"; the contract's final address depends only on bytecode hash + salt.
- **safe-global/safe-singleton-factory** — the EIP-155-safe variant for chains that reject chainless pre-signed txs (Celo, Avalanche, etc.). Per the safe-fndn README, the same deployer key (`(0xE1CB04A0fA36DdD16a06ea828007E35e1a3cBC37)`) yields factory address `(0x914d7Fec6aaC8cd542e72Bca78B30650d45643d7)` "for all bytecode-compatible EVM networks" (zkSync-EVM variant: `(0xaECdbB0a3B1C6D1Fe1755866e330D82eC81fD4FD)`); preinstalled on OP-Stack and zkSync-EVM chains. `(wilsoncusack/safe-singleton-deployer-sol)` notes Safe "has currently deployed this factory to 252 chains and it has the same address on 248." Submit a PR/issue to add a new chain's signed deployment tx.
- **CreateX** and OpenZeppelin's `(Create2Deployer)` are richer wrappers (CREATE/CREATE2/CREATE3, salt/sender/chainID protection).

A frontend "deploy to any chain" button = for each selected chain: switch chain → send the CREATE2 factory call with your salt + init code → display the (identical) computed address. This pairs perfectly with your no-proxy stance: CREATE2 gives address-stability without upgradeability.

Arc Testnet context. Chain **5042002** is **Arc Testnet** (hex `(0x4cef52)`), Circle's EVM Layer-1 (Malachite consensus engine for deterministic sub-second finality, **USDC as the native gas token** per Circle's pressroom and The Block). Per Circle's official pressroom ("Circle Launches Arc Public Testnet"), Arc was first introduced **Aug 12, 2025** and the public testnet went live **Oct 28, 2025** with 100+ institutional participants including BlackRock, Visa, and HSBC. Its CAIP-2 id is `(eip155:5042002)` (confirmed by The Graph's supported-networks page). Primary explorer is **arcscan** (`(https://testnet.arcscan.app)`), official RPC `(https://rpc.testnet.arc.network)`; Blockscout is listed among Circle's infra partners. Add it to `(allchain.html)` as `(eip155:5042002)` with USDC native currency (watch the gas-token/decimals nuance in wallets).

7. Post-deploy feedback & verification

Live feedback UX. After sending the deploy tx: show the **tx hash** immediately (link to `(<explorer>/tx/<hash>)`), poll with `(publicClient.waitForTransactionReceipt({hash, confirmations}))` (viem) or `(tx.wait(n)) / (provider.waitForTransaction(hash, n))` (ethers) to display **confirmation count**, then read `(receipt.contractAddress)` and render the **deployed**

address with an explorer link. ethers'

`contract.deploymentTransaction()`/`waitForDeployment()` gives the same.

Verification (you use Blockscout). Blockscout exposes a v2 REST API:

- `GET /api/v2/smart-contracts/verification/config` (capability check)
- `POST /api/v2/smart-contracts/<address>/verification/via/flattened-code` (body: `compiler_version`, `license_type`, `source_code`, optimization, constructor args)
- also `.../via/standard-input` (Solidity standard-JSON — ideal from Foundry, which can emit standard-json input), `.../via/sourcify`, `.../via/multi-part`, `.../via/vyper-*`.

From Foundry directly: `forge verify-contract --verifier blockscout --verifier-url https://<instance>/api <address> <path:Name>` (and `--verifier sourcify --verifier-url ...`) for Sourcify; Etherscan via API key). Standard-JSON input verification is the most reliable because it reproduces remappings/optimizer settings exactly — and your `foundry.toml` already pins `solc_version`, `optimizer`, `evm_version`. Wire a "Verify" button that POSTs the standard-json + constructor args to the Blockscout instance for whatever chain you just deployed to.

8. x402 / Parsec payment-gating the deployer (optional)

Your stack uses GoPlausible's **x402-avm** ("Parsec") — an Algorand (AVM) implementation of Coinbase's x402 HTTP-402 payment protocol. Per GoPlausible's Algorand-x402 docs README, "Coinbase x402 PR #361 ... has been merged to Coinbase x402 repository"; the Algorand Foundation announced full operational status on **Feb 23, 2026**, with GoPlausible as the network's designated facilitator. The relevant piece for gating a deployer UI is `@x402-avm/paywall` + a framework middleware (`@x402-avm/express`)/`(hono)`/`(next)`: a protected route returns **HTTP 402** with an HTML paywall. Matching GoPlausible's `@x402-avm/express` example verbatim, a route `/api/premium/*` is configured with `{ scheme: 'exact', network: ALGORAND_TESTNET_CAIP2, price: '$0.10' }`; the paywall "connects to Algorand wallets (Pera, Defly, Lute) via `@txnlab/use-wallet`" per the `@x402-avm/paywall` docs, the user pays, and on settlement the middleware serves the gated resource. To token-gate "deploy," put the deployer's artifact-serving endpoint (or a "get deploy authorization" endpoint) behind the x402 middleware; the EVM deploy itself still happens client-side via the connected EVM wallet. This cleanly separates **payment (Algorand/x402)** from **execution (EVM CREATE2 deploy)**, and fits ERC-8004 agent-identity gating if you later want per-agent deploy rights. `(GitHub)`

Details — Minimal code patterns

(a) EIP-6963 wallet discovery (vanilla JS)

```
js
```

```
const providers = [];
window.addEventListener('eip6963:announceProvider', (e) => {
  providers.push(e.detail); // {info:{uuid,name,icon,rdns}, provider}
  renderWalletButton(e.detail);
});
window.dispatchEvent(new Event('eip6963:requestProvider'));
// on click: const provider = chosen.provider; // EIP-1193
```

(b) ABI-driven form generation (vanilla JS, ethers v6)

```
js

function renderFunction(fn, contract, signerContract) {
  const inputs = fn.inputs.map(i =>
    `<input data-type="${i.type}" placeholder="${i.name}||'" ($${i.type})">`).join('')
  // on submit: collect values, coerce by type (BigInt for uint*, etc.)
  const args = collectArgs(formEl, fn.inputs);
  const isRead = fn.stateMutability === 'view' || fn.stateMutability === 'pure';
  return isRead ? await contract[fn.name](...args)
    : await (await signerContract[fn.name](...args)).wait();
}

abi.filter(x => x.type === 'function')
  .forEach(fn => mount(fn.stateMutability === 'view' || fn.stateMutability === 'pure'
    ? readPane : writePane, renderFunction(fn)));
```

(c) Deploy a Foundry artifact from the browser — ethers v6

```
js

const art = await (await fetch('/out/Counter.sol/Counter.json')).json();
const provider = new ethers.BrowserProvider(chosen.provider);
const signer = await provider.getSigner();
const factory = new ethers.ContractFactory(art.abi, art.bytecode.object, signer);
const contract = await factory.deploy(...ctorArgs); // wallet pops up
const tx = contract.deploymentTransaction();
showTx(`${explorer}/tx/${tx.hash}`);
await contract.waitForDeployment();
showAddress(await contract.getAddress());
```

(d) Deploy — viem (browser wallet)

```
js
```



```
import {createWalletClient, createPublicClient, custom, http} from 'viem';
const wallet = createWalletClient({ transport: custom(chosen.provider) });
const pub = createPublicClient({ chain, transport: http() });
const [account] = await wallet.getAddresses();
const hash = await wallet.deployContract({ abi: art.abi, bytecode: art.bytecode.object });
const {contractAddress} = await pub.waitForTransactionReceipt({ hash, confirmations: 1 });
```

(e) Add/switch chain from your CAIP-2 map

```
js

async function ensureChain(p, entry) { // entry from allchain.html
  const chainId = '0x' + Number(entry.id).toString(16);
  try { await p.request({method: 'wallet_switchEthereumChain', params: [{chainId}]}); }
  catch (e) {
    if (e.code === 4902) {
      await p.request({method: 'wallet_addEthereumChain', params: [{
        chainId, chainName: entry.name, rpcUrls: entry.rpc,
        nativeCurrency: entry.nativeCurrency, blockExplorerUrls: entry.explorers
      }]});
    } else throw e;
  }
}
```

(f) Blockscout verification (standard-JSON, from a Verify button)

```
js

await fetch(`${blockscoutBase}/api/v2/smart-contracts/${address}/verification/via/standard-json`, {
  method: 'POST',
  headers: {'Content-Type': 'application/json'},
  body: JSON.stringify({
    compiler_version: 'v0.8.26+commit.8a97fa7a',
    license_type: 'apache-2.0',
    contract_name: 'Counter',
    files: { 'standard-input.json': standardJsonString }, // from forge build --standard-json
    constructor_args: encodedCtorArgs
  })
});
```

Recommended reference architecture (cypherpunk2048 conventions)

Stage 1 — Minimum viable deployer (vanilla, no framework). Plain HTML/JS page. EIP-6963 picker → `(BrowserProvider)/viem (walletClient)`. Ship Foundry artifacts (`(out/**/*.*.json)`) as static assets. Constructor form from ABI. `(ContractFactory.deploy) / (walletClient.deployContract)`. Tx-hash link → confirmation poll → deployed-address display. Apache 2.0, mainnet-only chain list, no proxy/admin-key code paths. This is buildable from the snippets above plus eth95/ethereum-abi-ui as references.

Stage 2 — Multi-chain + deterministic. Drive the chain switcher from `(allchain.html)` (CAIP-2 → hex chainId/RPC/explorer/native currency). Add a CREATE2 path via Safe singleton factory (`(@x914d7Fec6...)`) so BANKON/PYTHAI core contracts land at identical addresses on Base, Moonbeam, etc. Add a per-chain Blockscout "Verify" button (standard-JSON). Benchmark to advance: same computed address verified on ≥ 2 mainnets.

Stage 3 — Interaction console + payment gate. Add an abi.ninja/Debug-Contracts-style Read/Write console for deployed contracts (reuse the ABI form generator). Optionally front the artifact/authorization endpoint with x402-avm "Parsec" (Algorand paywall) for paid/agent-gated deploys; bind deploy rights to ERC-8004 agent identity if desired.

When to use React instead of vanilla: if you adopt scaffold-eth-2 or want the Wagmi-CLI Foundry plugin's generated type-safe hooks. Both keep Foundry as the build tool; neither requires Hardhat.

Caveats

- **No single canonical OSS repo perfectly implements "ship Foundry bytecode → deploy from wallet → live feedback → verify."** abi.ninja (interaction-only) and scaffold-eth-2 (backend deploy script) explicitly do not; CryptoFusion is the closest exact-pattern match but is a tutorial project without a confirmed canonical repo/license. Expect to assemble from the primitives (ethers `(ContractFactory)/viem (deployContract)`) rather than fork one project.
- **thirdweb and some token-factory dApps use proxies/upgradeable patterns and external infra** — audit any dependency against your no-proxy / no-admin-key / mainnet-only rules before adopting.
- **In-browser solc is heavy** (~8 MB compiler + ~500 KB wrapper, Worker-only) and introduces a non-deterministic compile step; for known contracts, shipping precompiled Foundry artifacts is safer and reproducible.
- **CREATE2 address-stability requires identical init code + salt + factory** on every chain; differing compiler settings (or constructor args baked into init code) change the address. Pin `(foundry.toml)` settings and keep constructor args identical across chains, or use CREATE3 to remove constructor-arg sensitivity.

- **Verification reliability:** flattened-source verification is fragile (compiler patch version, optimizer runs, import order); prefer Solidity standard-JSON input, which Foundry can emit and Blockscout accepts at `(.../via/standard-input)`.
- **Arc Testnet uses USDC as the gas token** and is a testnet; some wallet `wallet_addEthereumChain` flows assume 18-decimal native currency — validate the add-chain payload for non-standard native tokens. The Arc-specific deployer found `(xPOURY4/Arc-Token-Deployer)`, MIT) is a Python CLI, not a browser dApp.
- **Source dating/maturity:** ethereum-abi-ui and react-eth are old (years since last publish); their concepts are sound but copy the algorithm rather than depending on stale packages. abi.ninja, scaffold-eth-2, wagmi/viem, and the Blockscout API are current and actively maintained.
- **ERC-8004** (Identity Registry as ERC-721) is itself an **upgradeable** contract per its reference implementation — relevant only if you register agents, and a deliberate exception to your no-proxy rule that lives outside your own deploys.